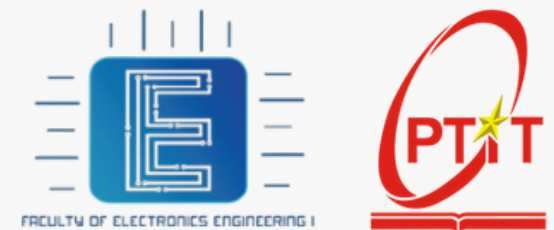


HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA KỸ THUẬT ĐIỆN TỬ 1

**Thiết kế và triển khai hệ thống AI Accelerator trên nền tảng
FPGA Xilinx Zynq UltraScale+ MPSoC
cho bài toán nhận dạng chữ số và chữ viết tay**

Giảng viên phụ trách: TS. Lê Minh Tuấn
ThS. Vương Viết Thao



Nội dung công việc - Dataflow & Scheduling

- Nghiên cứu và đề xuất cơ chế Dataflow phù hợp cho AI Accelerator trên FPGA nhằm tối ưu hiệu năng, giảm truy cập bộ nhớ và tăng khả năng tái sử dụng dữ liệu.
- Thiết kế và hiện thực mô-đun Sliding Window, cơ chế Data Reuse và Scheduler để điều phối luồng dữ liệu trong quá trình suy luận CNN.
- Phối hợp xây dựng kiến trúc kết nối giữa Memory Architecture và Compute Engine, đảm bảo dữ liệu được truyền và xử lý hiệu quả.
- Thực hiện mô phỏng, kiểm thử và đánh giá giải pháp theo các chỉ tiêu như độ trễ, thông lượng và hiệu quả sử dụng băng thông bộ nhớ.
- Phối hợp với các nhóm khác để tích hợp, kiểm thử và hoàn thiện toàn bộ hệ thống AI Accelerator.



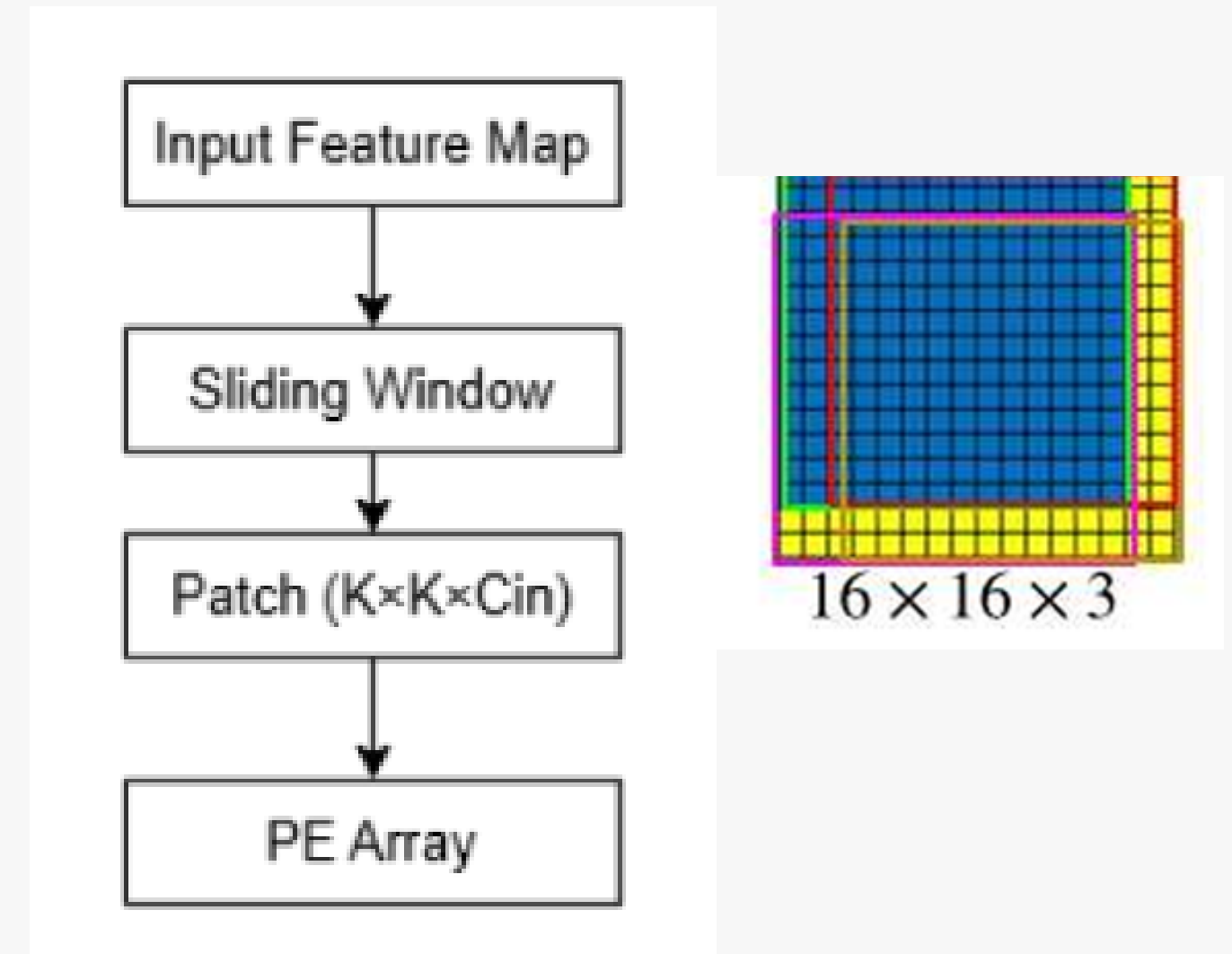
Tuần 1

Sliding Window Generator

1. Sliding Window là gì?

- Trong mạng CNN, phép tích chập (Convolution) yêu cầu mỗi đầu ra được tính từ một vùng dữ liệu cục bộ trên Input Feature Map.
- Ví dụ với Kernel 3×3 , mỗi Output Pixel cần một vùng dữ liệu 3×3 .
- Do đó, thay vì đọc toàn bộ ảnh mỗi lần tính toán, hệ thống sẽ tạo liên tục các cửa sổ (Patch) có kích thước bằng Kernel rồi đưa sang Compute Engine.

→ Cơ chế này gọi là Sliding Window.

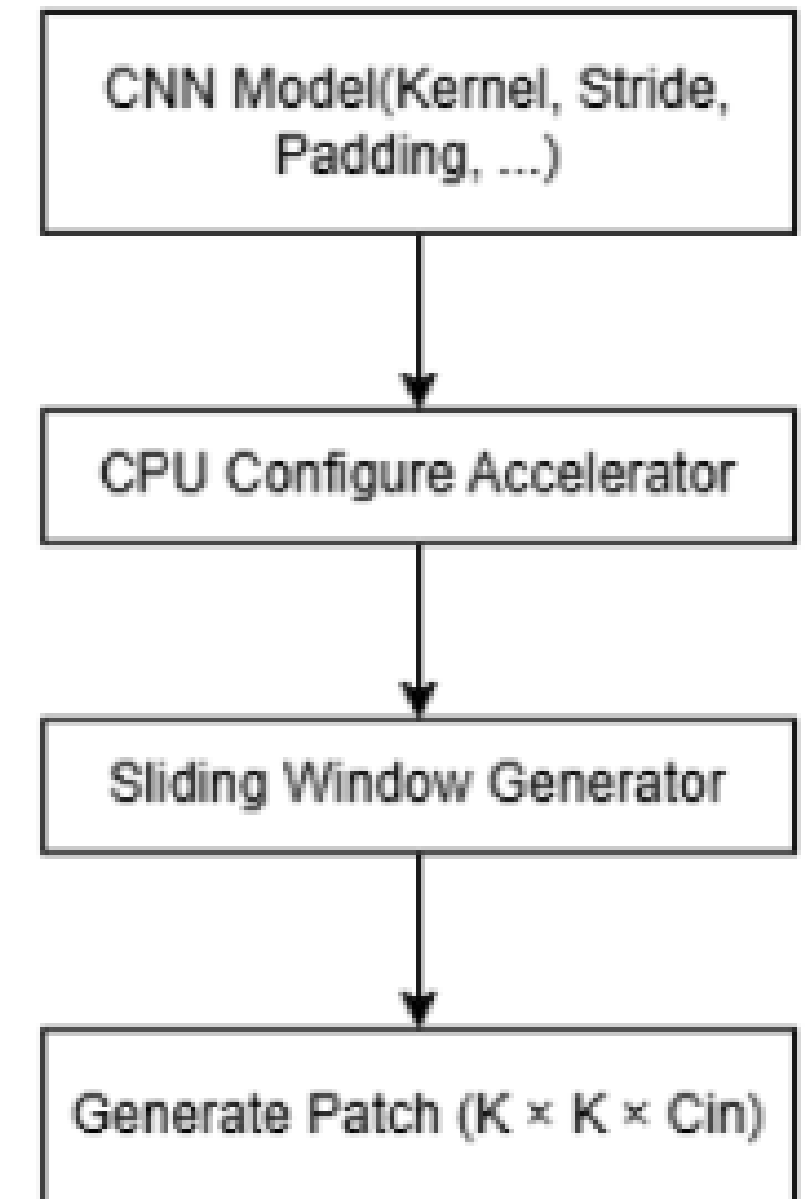


Sliding Window Generator

2. Thông số đầu vào của Sliding Window

- Sliding Window không tự quyết định cách quét ảnh.
- Toàn bộ thông số được xác định từ mô hình CNN đã thiết kế và được CPU hoặc Scheduler cấu hình trước khi bắt đầu suy luận.
- Các tham số này quyết định cách sinh Patch.

Tham số	Vai trò
Kernel Size ($K \times K$)	Xác định kích thước của mỗi Patch được tạo ra.
Stride (S)	Quy định số pixel cửa sổ trượt sau mỗi lần quét.
Padding (P)	Bổ sung vùng đệm ở biên của Input Feature Map.
Input Feature Map Size ($H \times W$)	Bổ sung vùng đệm ở biên của Input Feature Map.
Input Channels (C_{in})	Bổ sung vùng đệm ở biên của Input Feature Map.



Sliding Window Generator

3. Vì sao cần Sliding Window?

Nếu mỗi lần tính toán đều đọc lại toàn bộ dữ liệu từ bộ nhớ thì:

- Số lần truy cập BRAM/DDR tăng rất lớn.
- Băng thông bộ nhớ bị lãng phí.
- PE Array thường xuyên phải chờ dữ liệu.
- Throughput của hệ thống giảm.

→ Sliding Window giải quyết vấn đề này bằng cách chỉ đọc phần dữ liệu mới, còn dữ liệu cũ được tái sử dụng.

Ví dụ với Kernel 3x3

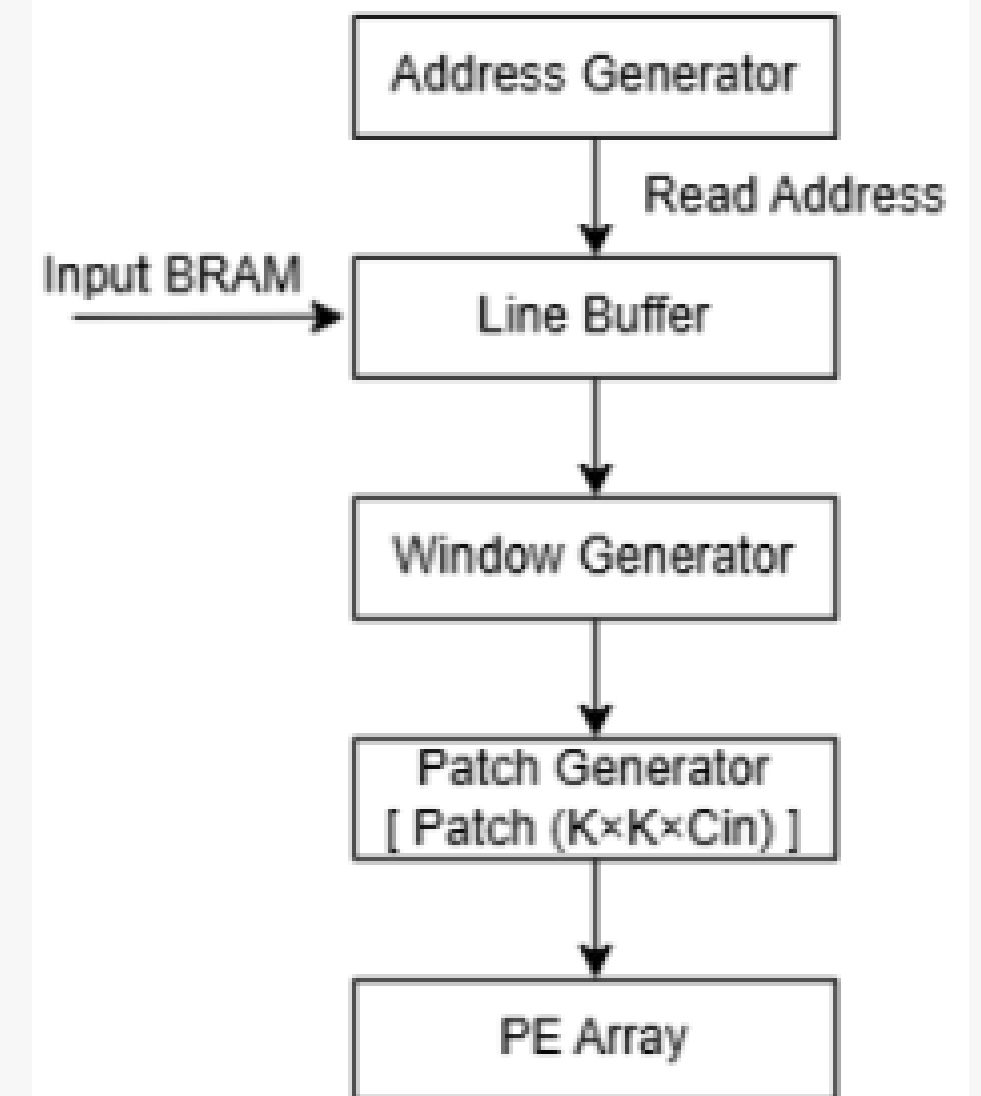
	1	2	3	4	5	6	7	8	9	10
1	1	2	3	4	5					
2	6	7	8	9	10					
3	11	12	13	14	15					
4	16	17	18	19	20					
5										
6	.	.	.	.	.	.	.	.	.	.
7										
8										
9										
10										

Sliding Window Generator

4. Kiến trúc bên trong Sliding Window

- Sliding Window gồm bốn khối chức năng chính.
- Mỗi khối đảm nhiệm một nhiệm vụ riêng trong quá trình tạo Patch.

Module	Chức năng
Address Generator	Sinh địa chỉ đọc dữ liệu từ Input BRAM dựa trên Kernel, Stride, Padding và kích thước Input Feature Map.
Line Buffer	Nhận dữ liệu từ Input BRAM, lưu các hàng dữ liệu và hỗ trợ Data Reuse giữa các Patch liên tiếp.
Window Generator	Trích xuất dữ liệu từ Line Buffer để tạo cửa sổ có kích thước $K \times K \times C_{in}$.
Patch Generator	Đóng gói cửa sổ thành Patch theo định dạng đầu vào của PE Array và xuất liên tục đến Compute Engine.



Sliding Window Generator

4. Kiến trúc bên trong Sliding Window

4.1 Address Generator

- Address Generator là khối điều khiển việc đọc dữ liệu từ Input BRAM.
- Nó sinh địa chỉ đọc dựa trên:
 - Kernel Size
 - Stride
 - Padding
 - Kích thước Feature Map

Mục tiêu là đảm bảo PE luôn nhận đúng Patch theo kiến trúc CNN.

Ví dụ: Stride = 1

$(0,0) \rightarrow (0,1) \rightarrow (0,2)$

Ví dụ: Stride = 2

$(0,0) \rightarrow (0,2) \rightarrow (0,4)$

→ Address Generator quyết định vị trí của từng Patch trên Feature Map.

Sliding Window Generator

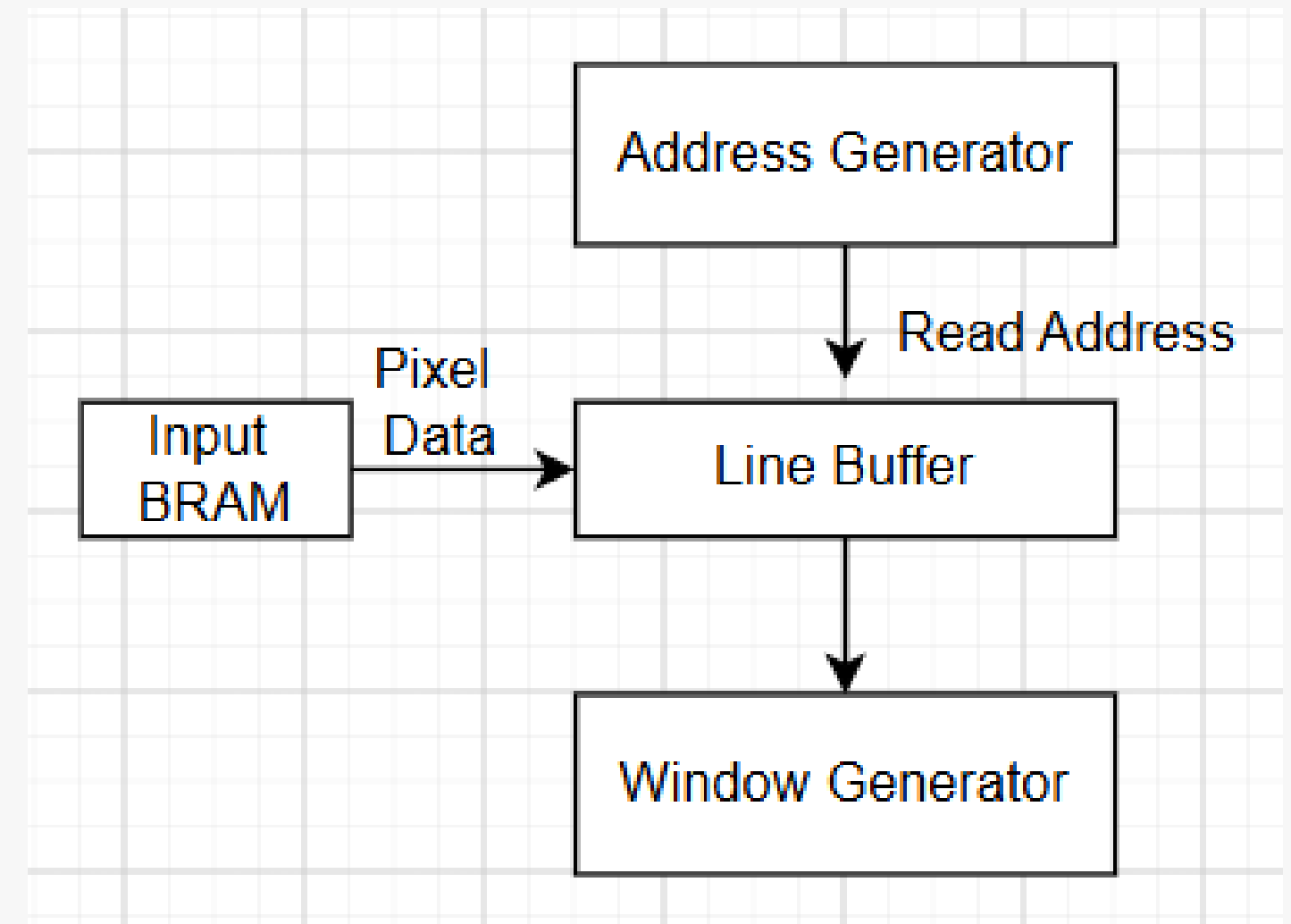
4. Kiến trúc bên trong Sliding Window

4.2 Line Buffer

- Line Buffer là bộ nhớ đệm lưu các hàng dữ liệu vừa đọc từ Input BRAM.
- Do các Patch liên tiếp có nhiều vùng chồng lấp, phần lớn dữ liệu của Patch trước có thể được sử dụng lại cho Patch sau.
- Nhờ đó, hệ thống không cần đọc lại toàn bộ dữ liệu từ BRAM.

Lợi ích

- Giảm số lần đọc BRAM.
- Hỗ trợ Data Reuse.
- Tăng Throughput.



Sliding Window Generator

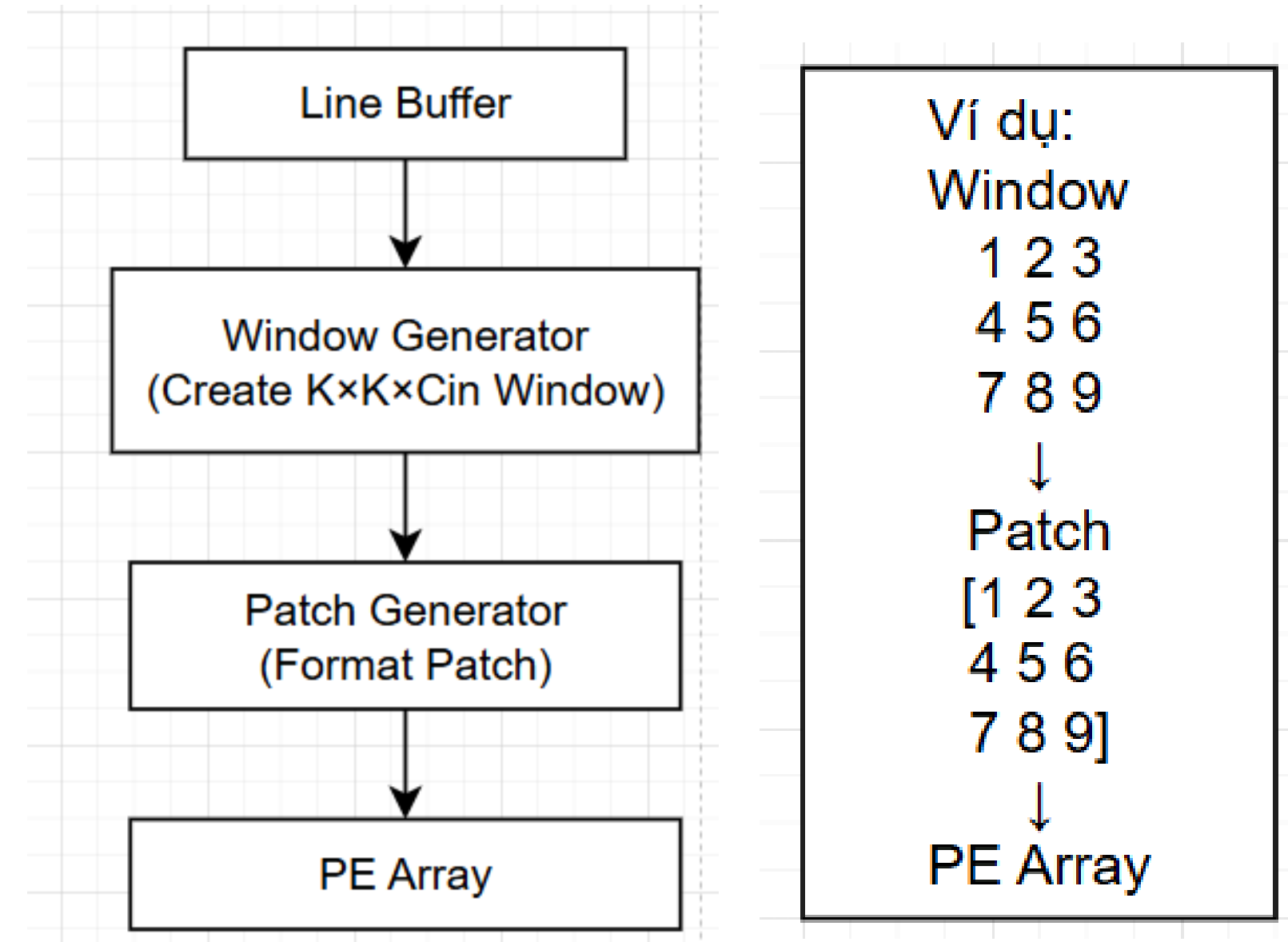
4. Kiến trúc bên trong Sliding Window

4.3 Window Generator & Patch Generator

- Sau khi dữ liệu được lưu trong Line Buffer, Window Generator sẽ trích xuất vùng dữ liệu có kích thước $K \times K \times C_{in}$.
- Tiếp theo, Patch Generator đóng gói cửa sổ này thành định dạng đầu vào của PE Array.

Kết luận

- Window Generator tạo cửa sổ $K \times K \times C_{in}$.
- Patch Generator chuyển cửa sổ thành dữ liệu đầu vào của PE Array.
- Hai khối phối hợp để cấp Patch liên tục cho Compute Engine.



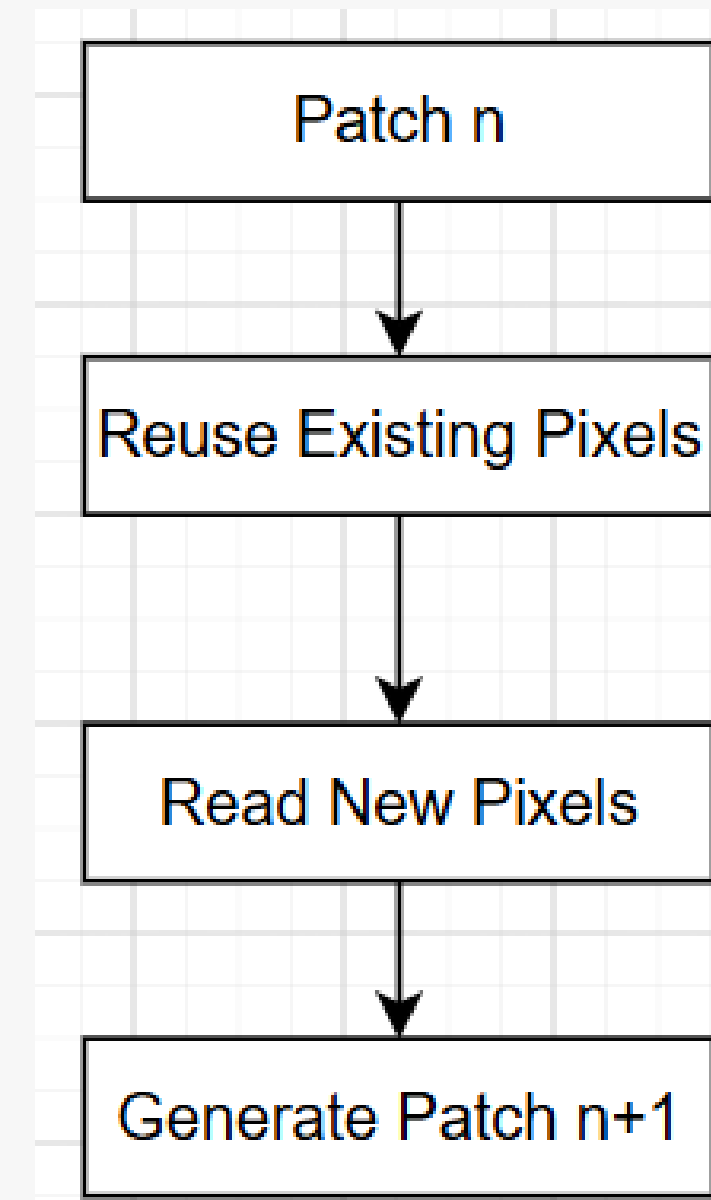
Sliding Window Generator

5. Data Reuse

- Data Reuse là kỹ thuật tái sử dụng dữ liệu giữa các Patch liên tiếp trong quá trình quét Feature Map.
- Do các Patch liên tiếp có nhiều vùng dữ liệu chồng lấp, hệ thống không cần đọc lại toàn bộ dữ liệu từ Input BRAM. Thay vào đó, chỉ những Pixel mới được nạp vào Line Buffer để tạo Patch tiếp theo.
- Nhờ cơ chế này, lượng truy cập bộ nhớ được giảm đáng kể, đồng thời Compute Engine luôn được cung cấp dữ liệu liên tục.

Lợi ích

- Chỉ đọc thêm dữ liệu mới.
- Giảm băng thông BRAM/DDR.
- Giữ PE Array luôn có dữ liệu.
- Tăng Throughput.



Patch 1:
1 2 3
4 5 6
7 8 9

Patch 2:
2 3 4
5 6 7
8 9 10

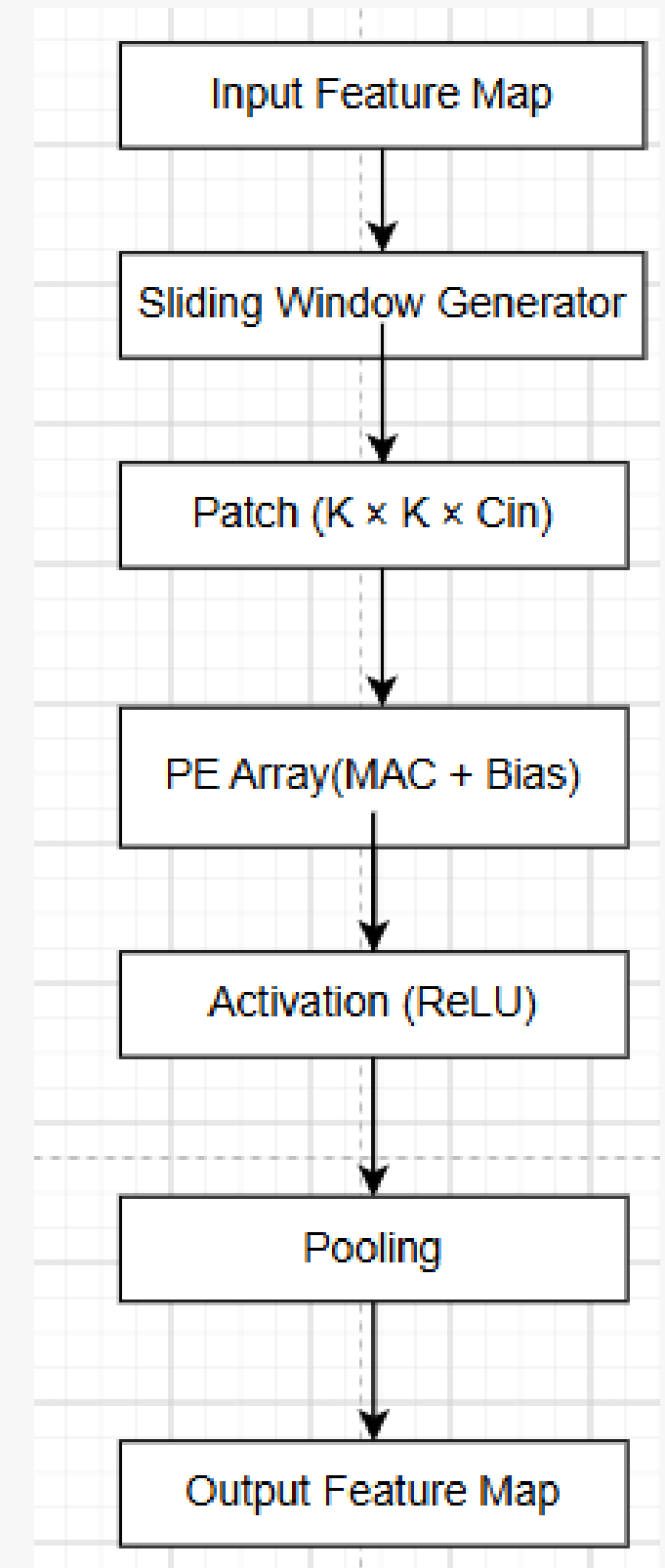
Giữ lại:
2 3
5 6
8 9

Đọc mới:
4
7
10

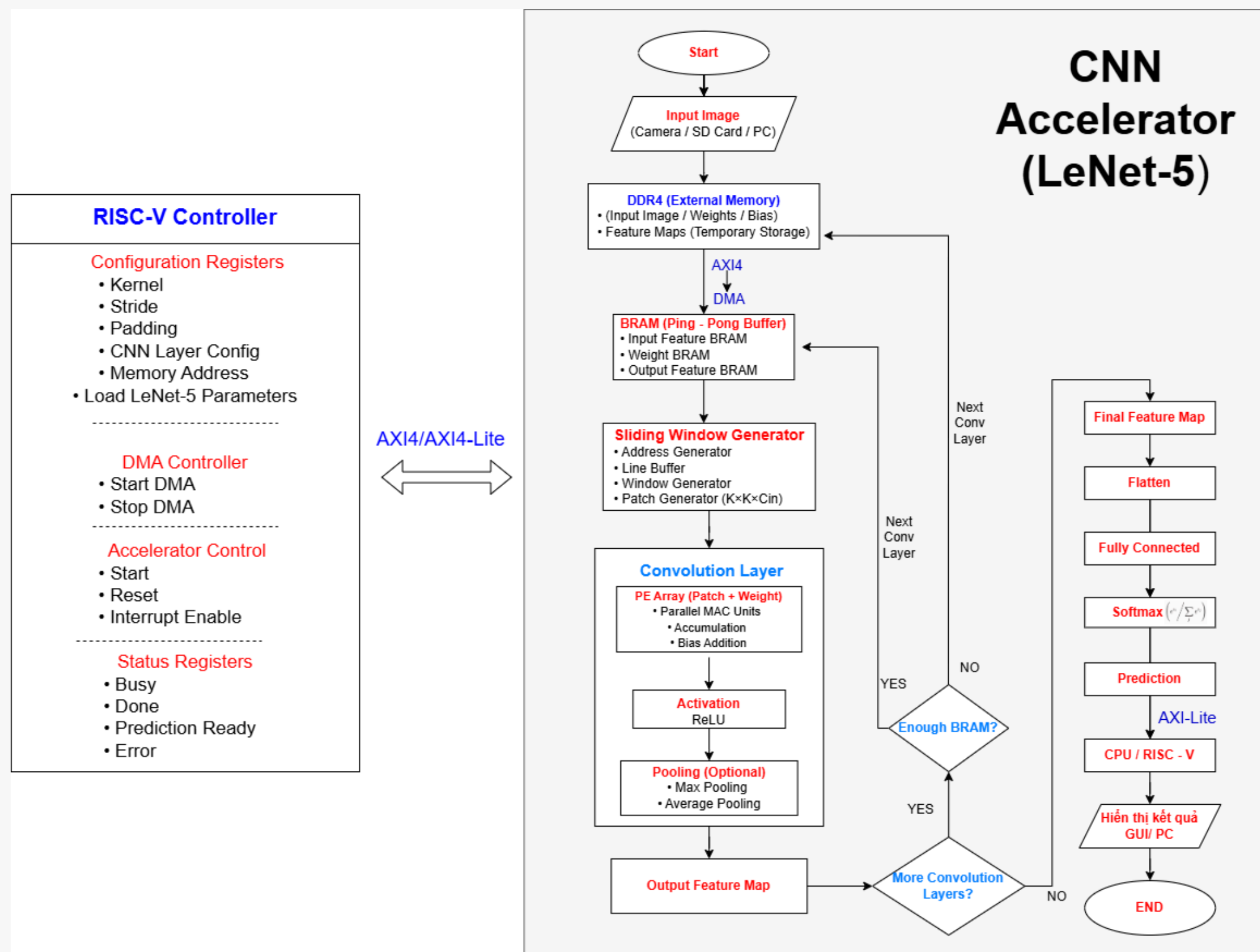
Sliding Window Generator

6. Vai trò của Sliding Window trong AI Accelerator

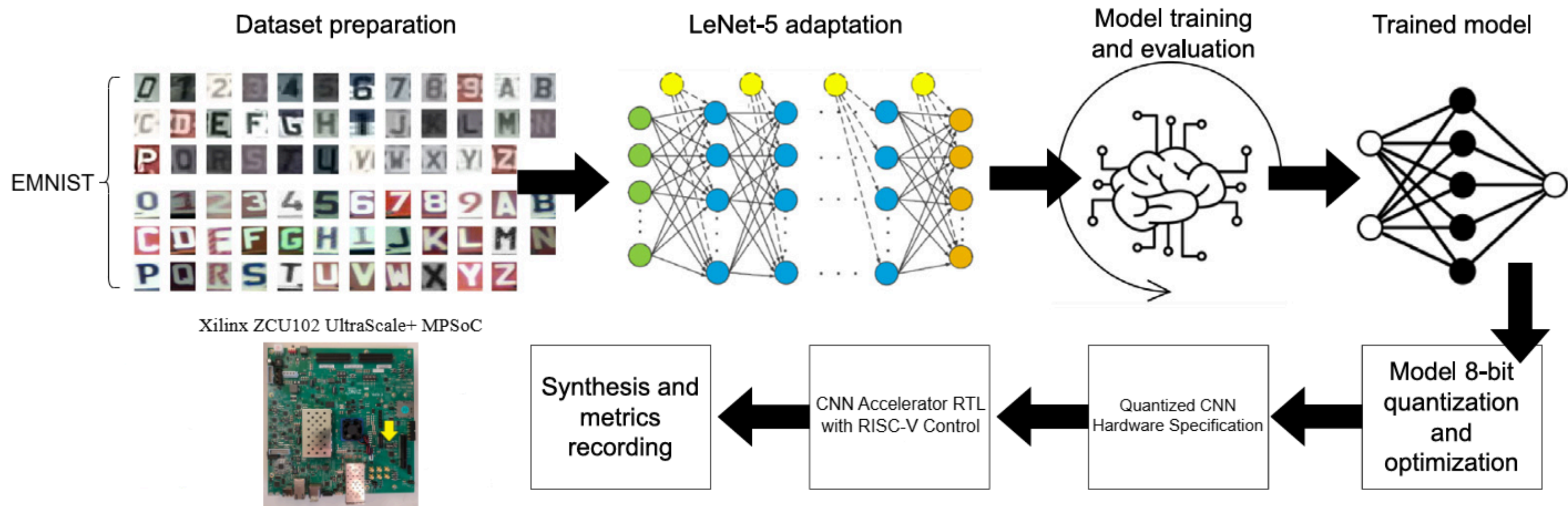
- Trích xuất Patch ($K \times K \times C_{in}$) từ Input Feature Map.
 - Sinh Patch theo các tham số Kernel, Stride và Padding.
 - Cung cấp dữ liệu đầu vào liên tục cho PE Array.
 - Hỗ trợ Data Reuse, giảm số lần truy cập BRAM/DDR.
 - Giúp Compute Engine hoạt động liên tục, hạn chế thời gian chờ dữ liệu.
 - Nâng cao Throughput và hiệu quả của AI Accelerator.
- Sliding Window là cầu nối giữa Memory Architecture và Compute Engine, đảm bảo dữ liệu được cấp liên tục và đúng định dạng cho quá trình Convolution.



Sơ đồ thuật toán



Sơ đồ kiến trúc tổng quát

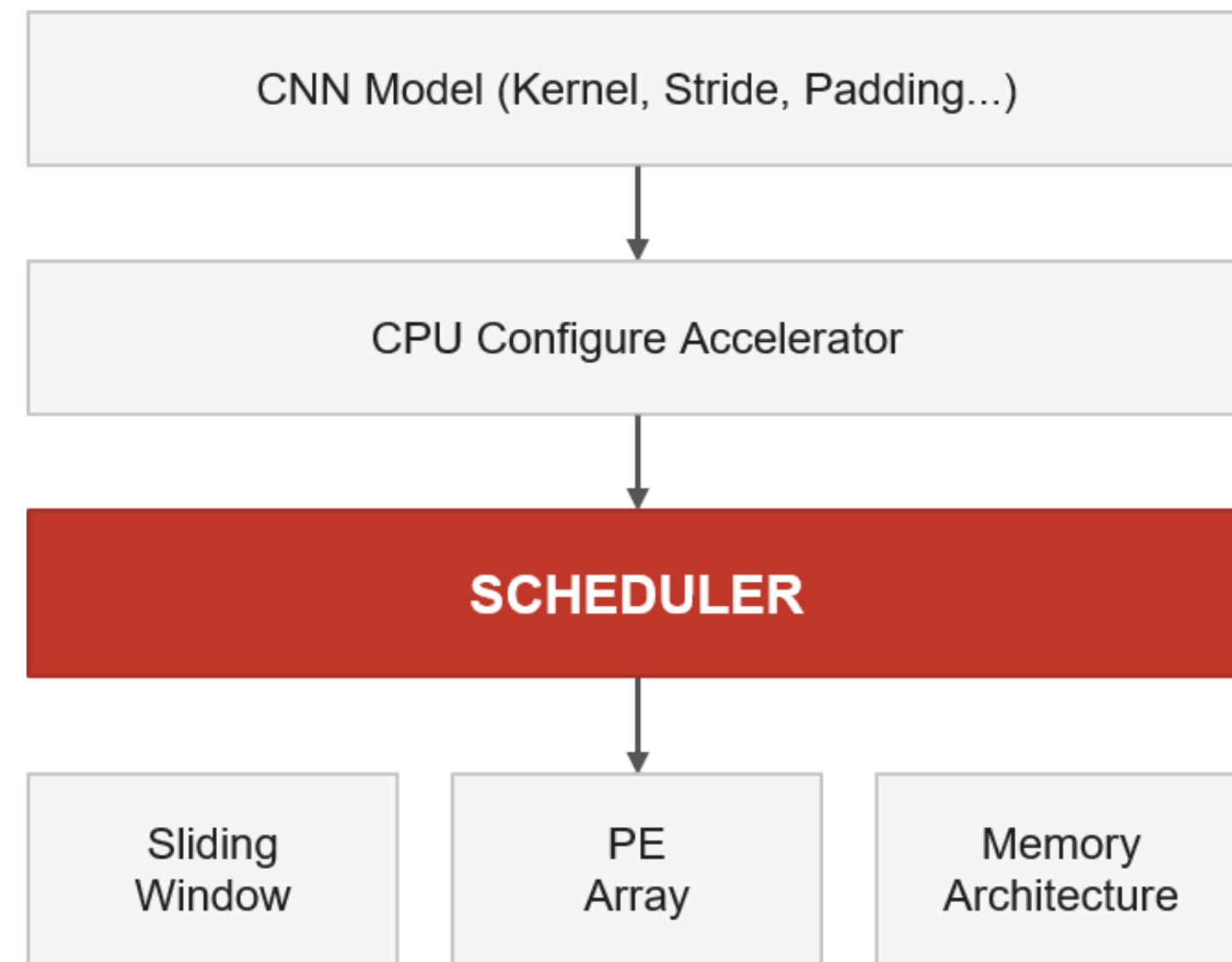


→ Mô hình LeNet-5 được huấn luyện và lượng tử hóa xuống 8-bit, sau đó được ánh xạ thành kiến trúc phần cứng và hiện thực bằng Verilog trên FPGA Xilinx ZCU102 để tổng hợp và đánh giá hiệu năng.

Scheduler

1. Scheduler là gì?

- Scheduler là khối điều khiển trình tự thực thi của toàn bộ quá trình suy luận CNN trên Accelerator.
- Nhận cấu hình lớp mạng (Layer Config) từ RISC-V Controller thông qua Configuration Registers.
- Quyết định khi nào và theo thứ tự nào các khối Sliding Window, PE Array, Activation, Pooling được kích hoạt.
- Là lớp điều phối trung tâm, kết nối giữa phần điều khiển phần mềm (RISC-V) và phần thực thi phần cứng (Dataflow, Compute Engine).



→ **Scheduler là "nhạc trưởng" điều phối toàn hệ thống, đảm bảo dữ liệu và tính toán diễn ra đồng bộ.**

2. Vì sao cần Scheduler?

Nếu không có cơ chế điều phối tập trung:

- Các mô-đun (Sliding Window, PE Array, Pooling...) có thể tranh chấp BRAM cùng lúc.
- Không biết khi nào lớp Convolution hiện tại xong để chuyển sang lớp tiếp theo.
- Dễ xảy ra tình trạng đọc dữ liệu chưa sẵn sàng (race condition).
- PE Array thường xuyên phải rảnh rồi chờ dữ liệu.
- Throughput của toàn hệ thống giảm.

Không có Scheduler

Tranh chấp BRAM

Không rõ trạng thái lớp

Race condition dữ liệu

PE Array chờ dữ liệu

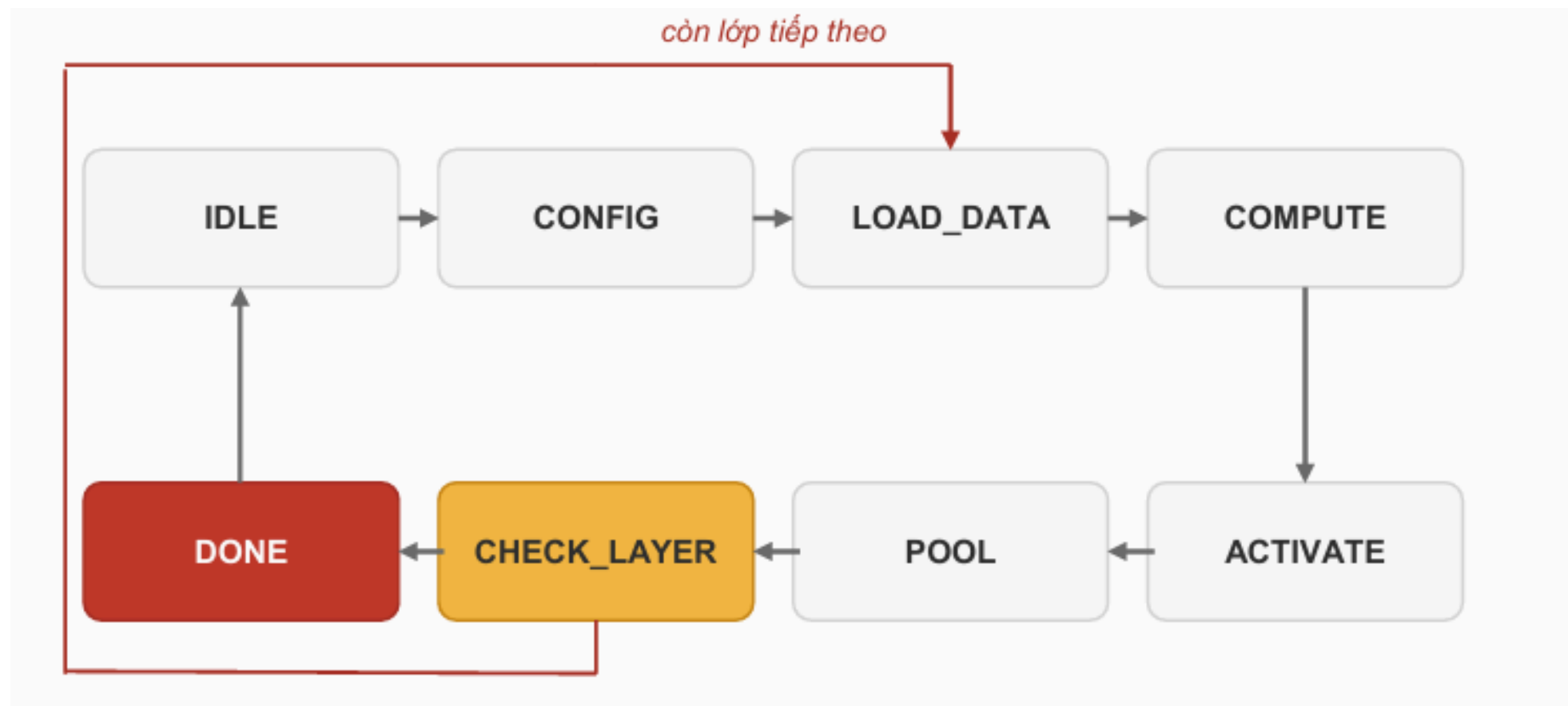
Throughput giảm

→ **Scheduler giải quyết bằng cách chuẩn hoá một chu trình điều khiển thống nhất cho toàn hệ thống.**

3. Kiến trúc bên trong Scheduler

Module	Chức năng
Layer Config Decoder	Đọc thông số lớp hiện tại (loại lớp, kernel, stride, địa chỉ bộ nhớ) từ Configuration Registers do RISC-V nạp vào.
FSM điều khiển chính	Quản lý trạng thái toàn hệ thống: IDLE → LOAD → COMPUTE → POOLING → DONE → NEXT_LAYER.
Handshake Controller	Trao đổi tín hiệu valid/ready giữa Sliding Window, PE Array và bộ nhớ để đồng bộ dữ liệu.
Buffer Manager (Ping-Pong Control)	Quyết định thời điểm chuyển đổi giữa 2 vùng BRAM (ping/pong) để nạp dữ liệu lớp sau song song với tính lớp hiện tại.
Layer Sequencer	Xác định lớp tiếp theo cần xử lý (Conv → Activation → Pooling → FC...) theo kiến trúc LeNet-5.

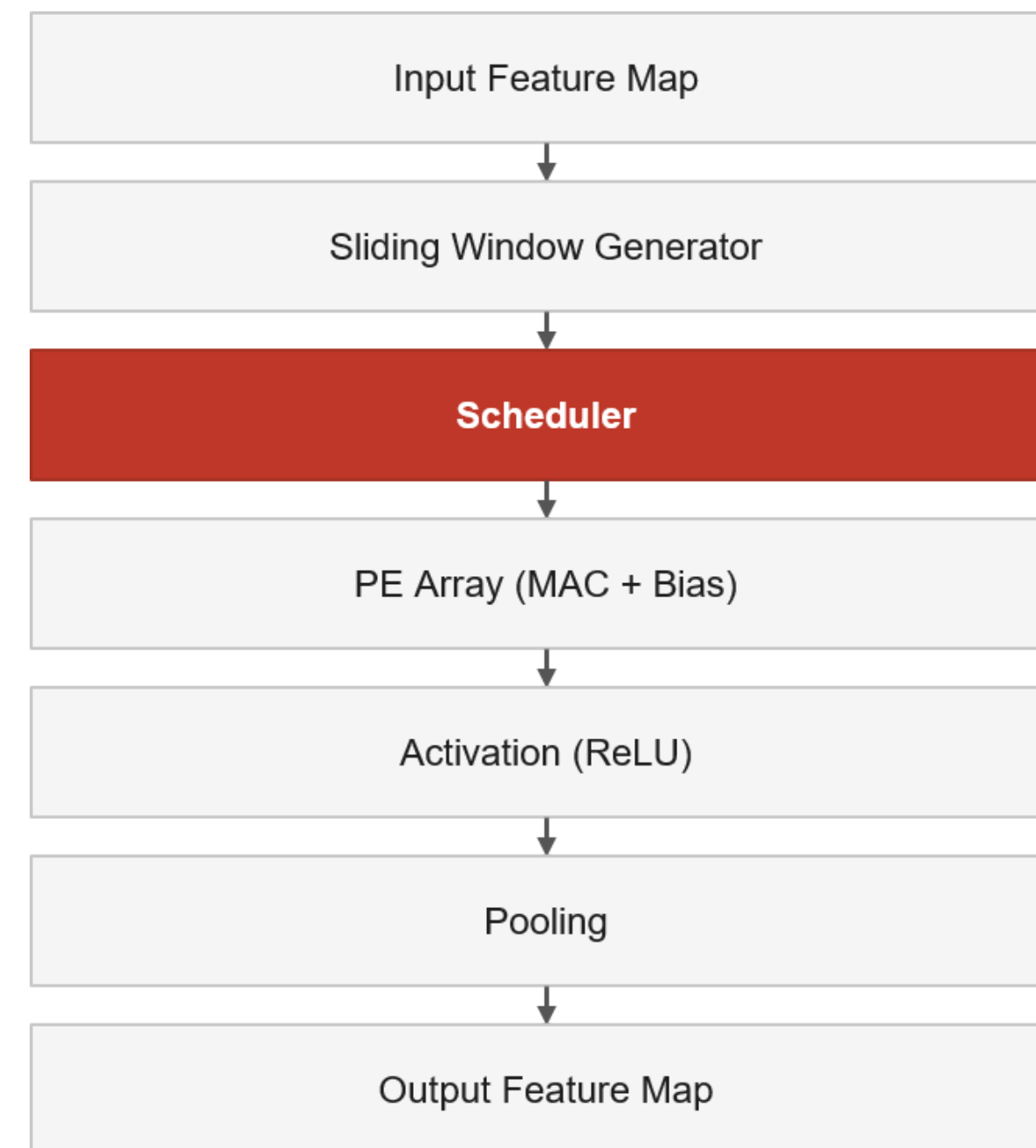
4. Chu trình hoạt động của Scheduler (FSM)



- Mỗi trạng thái tương ứng với một giai đoạn trong sơ đồ hệ thống tổng thể (Sliding Window → Convolution → Activation → Pooling → Output Feature Map).
- Scheduler dùng các thanh ghi trạng thái sẵn có (Busy, Done, Prediction Ready, Error) để giao tiếp ngược lại với RISC-V Controller.
- Trạng thái DONE báo hiệu Prediction Ready, kết thúc một chu trình suy luận.

5. Vai trò của Scheduler trong AI Accelerator

- Đồng bộ hoạt động giữa Sliding Window Generator, PE Array và Memory Architecture.
- Cho phép overlap giữa nạp dữ liệu và tính toán (Double Buffering) để tăng Throughput.
- Đảm bảo chuyển tiếp đúng thứ tự giữa các lớp CNN (Conv1 → Pool1 → Conv2 → Pool2 → FC → Softmax).
- Là cầu nối giữa lớp điều khiển phần mềm (RISC-V) và lớp thực thi phần cứng (Dataflow + Compute Engine).



Đánh giá Row-Stationary

1. Ưu / nhược điểm chi tiết từng phương án Dataflow

Dataflow	Ưu điểm chính	Hạn chế chính
Weight Stationary (WS)	Tái sử dụng Weight cao, đơn giản khi hiện thực.	Tốn băng thông cho Input và Partial Sum.
Output Stationary (OS)	Giảm ghi/đọc Partial Sum ra ngoài.	Tái sử dụng Weight và Input thấp.
Input Stationary (IS)	Tái sử dụng Input Feature Map cao, phù hợp Sliding Window.	Weight phải nạp lại nhiều lần.
Row Stationary (RS)	Cân bằng tái sử dụng cả 3 loại dữ liệu, giảm truy cập DDR4 mạnh nhất.	Độ phức tạp điều khiển cao, cần Scheduler + Line Buffer phối hợp chặt.

→ *Row-Stationary là phương án cân bằng nhất, phù hợp với hạ tầng Line Buffer đã có của hệ thống.*



2. Vì sao Row-Stationary phù hợp với hệ thống

- Hệ thống đã có sẵn Line Buffer trong Sliding Window Generator – thành phần cần thiết để RS khai thác tái sử dụng theo hàng (row).
- Mô hình LeNet-5 dùng kernel nhỏ (5×5), nhiều lớp Convolution nối tiếp → RS tận dụng tốt sự chồng lấp giữa các Patch liên tiếp.
- ZCU102 có tài nguyên BRAM/URAM đủ lớn để chấp nhận độ phức tạp điều khiển cao hơn của RS, đổi lại giảm mạnh số lần truy cập DDR4.

3. Đánh đổi (trade-off) cần lưu ý khi hiện thực

- Chi phí điều khiển: Scheduler và Address Generator phải theo dõi đồng thời 3 loại dữ liệu (Input, Weight, Partial Sum) trong cùng một PE.
- Chi phí phần cứng: cần nhiều thanh ghi/bộ đệm cục bộ hơn trong mỗi PE → ảnh hưởng mức sử dụng LUT/FF khi tổng hợp trên FPGA.
- Phạm vi áp dụng: với quy mô LeNet-5 nhẹ, nhóm áp dụng RS ở mức “định hướng” – dùng Line Buffer để tái sử dụng theo hàng, không cần PE Array phức tạp như kiến trúc Eyeriss gốc.

4. Tiêu chí đánh giá thực nghiệm

Chỉ tiêu	Cách đo
Số lần truy cập DDR4	Đếm số giao dịch AXI4 qua DMA trong 1 lần suy luận.
Tỷ lệ tái sử dụng dữ liệu	So sánh số Pixel đọc mới / tổng số Pixel cần cho Patch.
Độ trễ (Latency)	Thời gian từ Input Image đến Final Feature Map.
Thông lượng (Throughput)	Số ảnh xử lý được / giây.
Mức sử dụng tài nguyên FPGA	LUT, FF, BRAM, URAM sau Synthesis / Implementation.

→ **Kết quả thực nghiệm sẽ đối chiếu với bảng so sánh lý thuyết để xác nhận Row-Stationary có thực sự vượt trội trong hệ thống cơ thể này.**

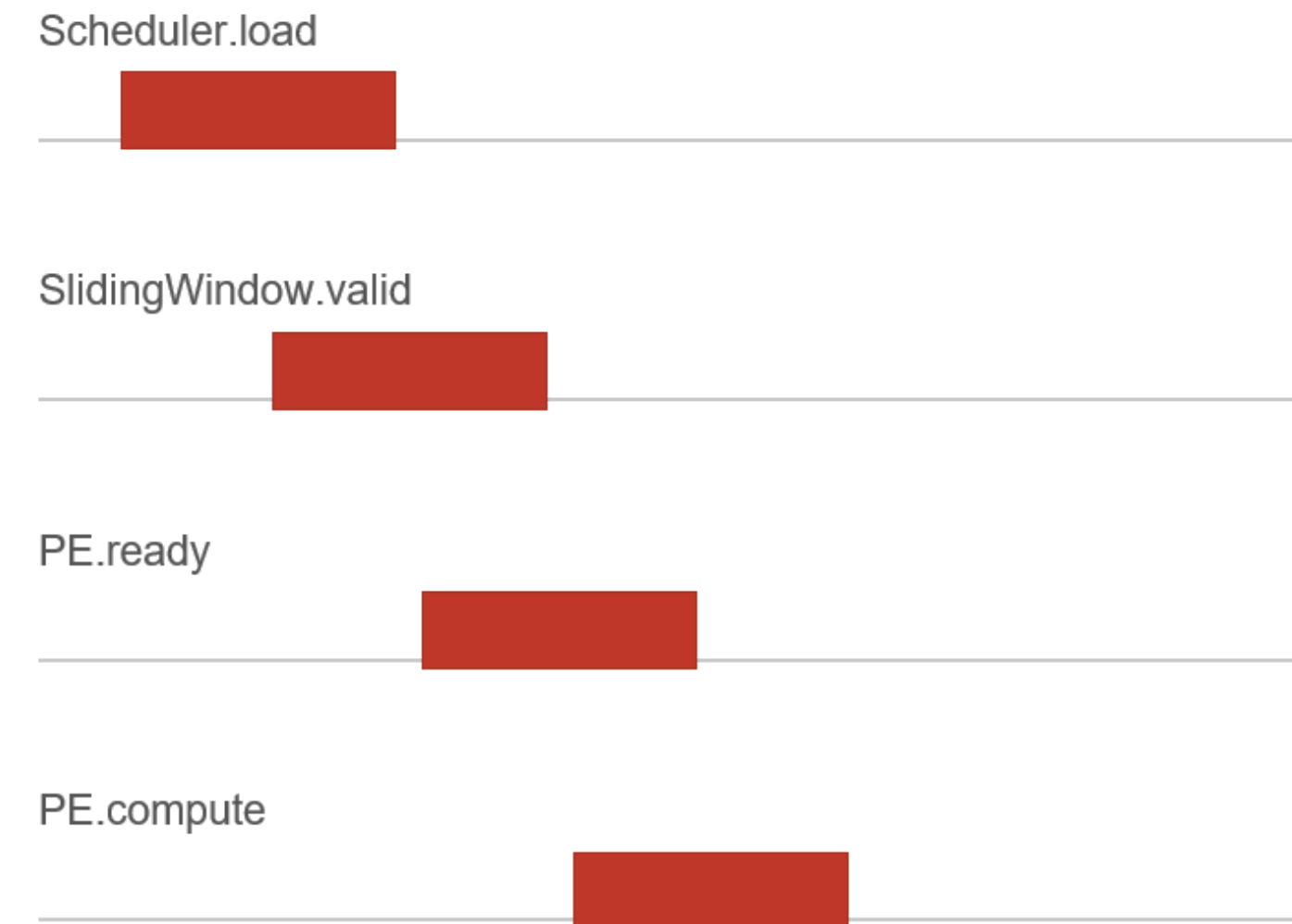
5. Pseudocode / Bảng chuyển trạng thái FSM

Trạng thái	Điều kiện vào	Hành động	Kế tiếp
IDLE	Reset / Start=0	Chờ tín hiệu Start	CONFIG
CONFIG	Start=1	Đọc Layer Config (Kernel, Stride, Padding)	LOAD_DATA
LOAD_DATA	Config hợp lệ	Kích hoạt Sliding Window đọc Patch	COMPUTE
COMPUTE	Patch valid=1	PE Array: MAC + Bias	ACTIVATE
ACTIVATE	Conv xong	Áp dụng ReLU	POOL
POOL	Có Pooling	Max/Average Pooling	CHECK_LAYER
CHECK_LAYER	1 vòng xong	Kiểm tra còn Conv Layer?	LOAD_DATA / DONE
DONE	Hết các lớp	Ghi Prediction Ready	IDLE

→Bảng trạng thái này là input trực tiếp để hiện thực FSM bằng Verilog/VHDL hoặc Vitis HLS ở Milestone M2.

6. Giao tiếp Handshake với các phân hệ khác

- Với Sliding Window Generator: Scheduler cấu hình Address Generator (Kernel, Stride, Padding), nhận tín hiệu Patch valid trước khi chuyển dữ liệu sang PE Array.
- Với Compute Engine (Nhóm 1): đồng bộ thời điểm PE Array nhận Patch và bắt đầu tính MAC, tránh đọc dữ liệu chưa ổn định.
- Với Memory Architecture (Nhóm 2): điều khiển cơ chế Ping-Pong Buffer, quyết định thời điểm chuyển vùng BRAM.
- Với System Integration (Nhóm 3): cung cấp tín hiệu Busy / Done / Error để RISC-V giám sát qua AXI4-Lite.



→ *Chuỗi tín hiệu valid/ready tuần tự đảm bảo mỗi phân hệ chỉ xử lý khi dữ liệu từ phân hệ trước đã sẵn sàng.*



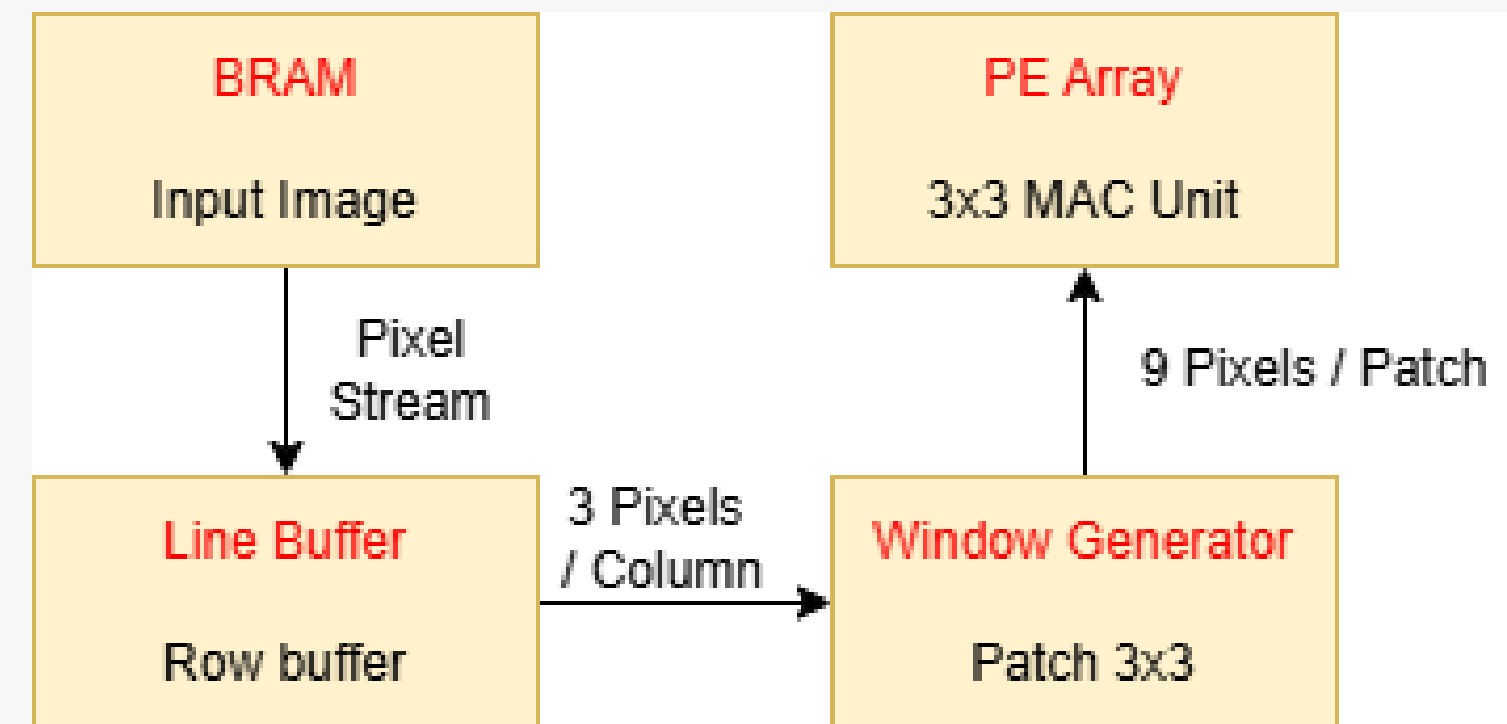
Tuần 2

Sliding Window Generator

1. Vị trí của Line Buffer trong hệ thống

Line Buffer là tầng trung gian giữa bộ nhớ ảnh và Window Generator

- Nhận dữ liệu ảnh theo dạng pixel stream
- Lưu lại các hàng ảnh trước đó
- Cung cấp đồng thời 3 pixel thuộc cùng một cột (3 hàng)
- Cho phép Window Generator xây dựng cửa sổ trượt 3×3



Sliding Window Generator

1.2. Nguyên lý hoạt động của Line Buffer

Ví dụ hoạt động của Line Buffer

Ban đầu:

Row0 : 0 1 2 3 4 5

Row1 : 10 11 12 13 14 15

Row2 : 20 21 22 23 24 25

Khi dữ liệu được nạp theo từng cột:

Input Stream: 20 → 21 → 22 → 23 → ...

Line Buffer xuất đồng thời:

(0,10,20)

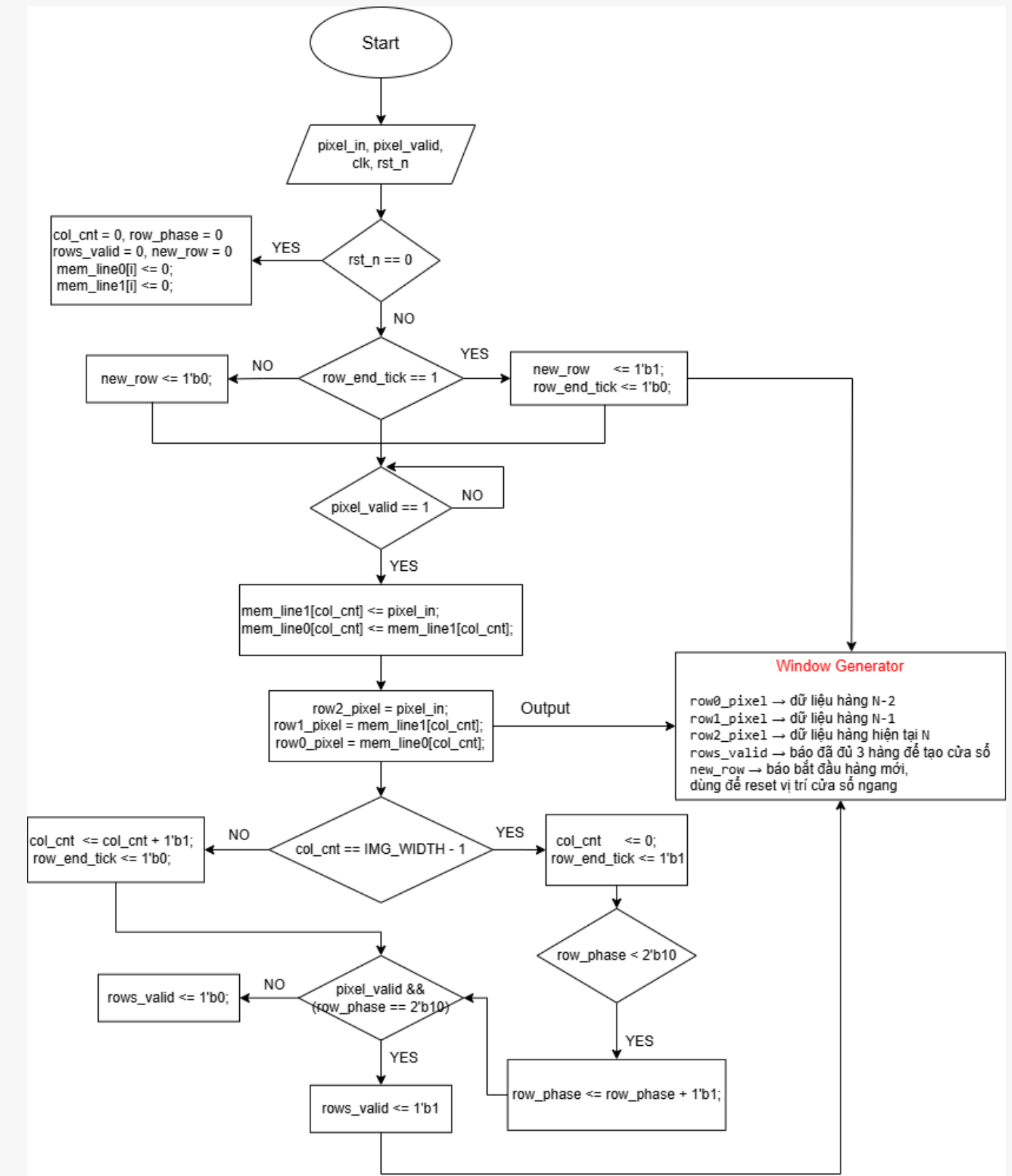
(1,11,21)

(2,12,22)

(3,13,23)

...

→ Mỗi chu kỳ chỉ nạp 1 pixel mới, trong khi hai hàng còn lại được tái sử dụng, tạo dữ liệu theo chiều dọc cho Window Generator.



Sliding Window Generator

1.2. Nguyên lý hoạt động của Line Buffer

Ví dụ đến clock nhận pixel 20 (Row 2)

- Input:
pixel_in = 20
- Line Buffer xuất:
row0_pixel = 0
row1_pixel = 10
row2_pixel = 20
- Gửi sang Window Generator:
[0
10
20]
- Reset cột khi đến cột của patch cuối

5x5	Col 0	Col 1	Col 2	Col 3	Col 4
Row 0	0	1	2	3	4
Row 1	10	11	12	13	14
Row 2	20	21	22	23	24
Row 3	30	31	32	33	34
Row 4	40	41	42	43	44

Giai đoạn 1: Nhận và lưu Row 0

0 → 1 → 2 → 3 → 4

Giai đoạn 2: Nhận và lưu Row 1

10 → 11 → 12 → 13 → 14

Pixel_in

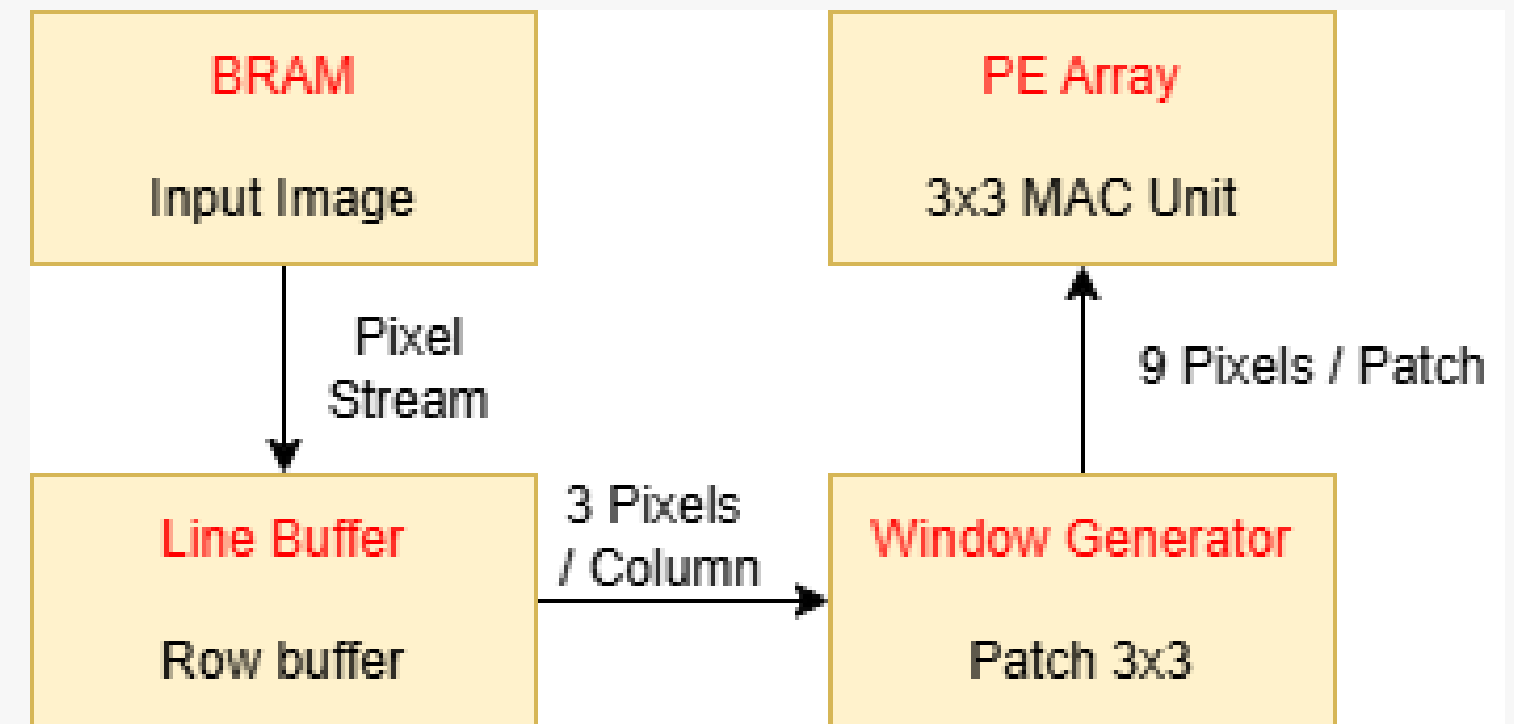
mem_1

mem_0

Sliding Window Generator

2. Vai trò của Window Generator

- Là cầu nối giữa Memory Architecture và Compute Engine.
- Nhận dữ liệu pixel theo dạng stream từ Line Buffer.
- Chuyển đổi dữ liệu tuần tự thành patch 3×3 song song.
- Cung cấp dữ liệu đúng định dạng và đúng thời điểm cho PE Array.
- Duy trì tốc độ truyền dữ liệu liên tục cho khối tính toán.



Sliding Window Generator

2.1. Window Generator

Sẽ nhận input từ Line Buffer:

Mỗi chu kỳ clock, Window Generator nhận 3 pixel cùng một cột:

Ví dụ:

	Col 0	Col 1	Col 2	Col 3	Col 4
Row 0	0	1	2	3	4
Row 1	10	11	12	13	14
Row 2	20	21	22	23	24

Line Buffer xuất tuần tự:

Cột 0 → (0, 10, 20)

Cột 1 → (1, 11, 21)

Cột 2 → (2, 12, 22)

Cột 3 → (3, 13, 23)

...

Mục tiêu của Window Generator

Chuyển:

(0,10,20)

(1,11,21)

(2,12,22)

thành:

0	1	2
10	11	12
20	21	22

→ Đây là một patch 3×3 hoàn chỉnh để gửi tới PE Array.

Sliding Window Generator

2.2. Nguyên lý hoạt động của Window Generator

- Nhận dữ liệu từ Line Buffer

Mỗi chu kỳ pixel_valid = 1, Window Generator nhận:

row0_pixel
row1_pixel
row2_pixel

Ba pixel này tạo thành một cột mới của cửa sổ.

- Dịch cửa sổ theo chiều ngang

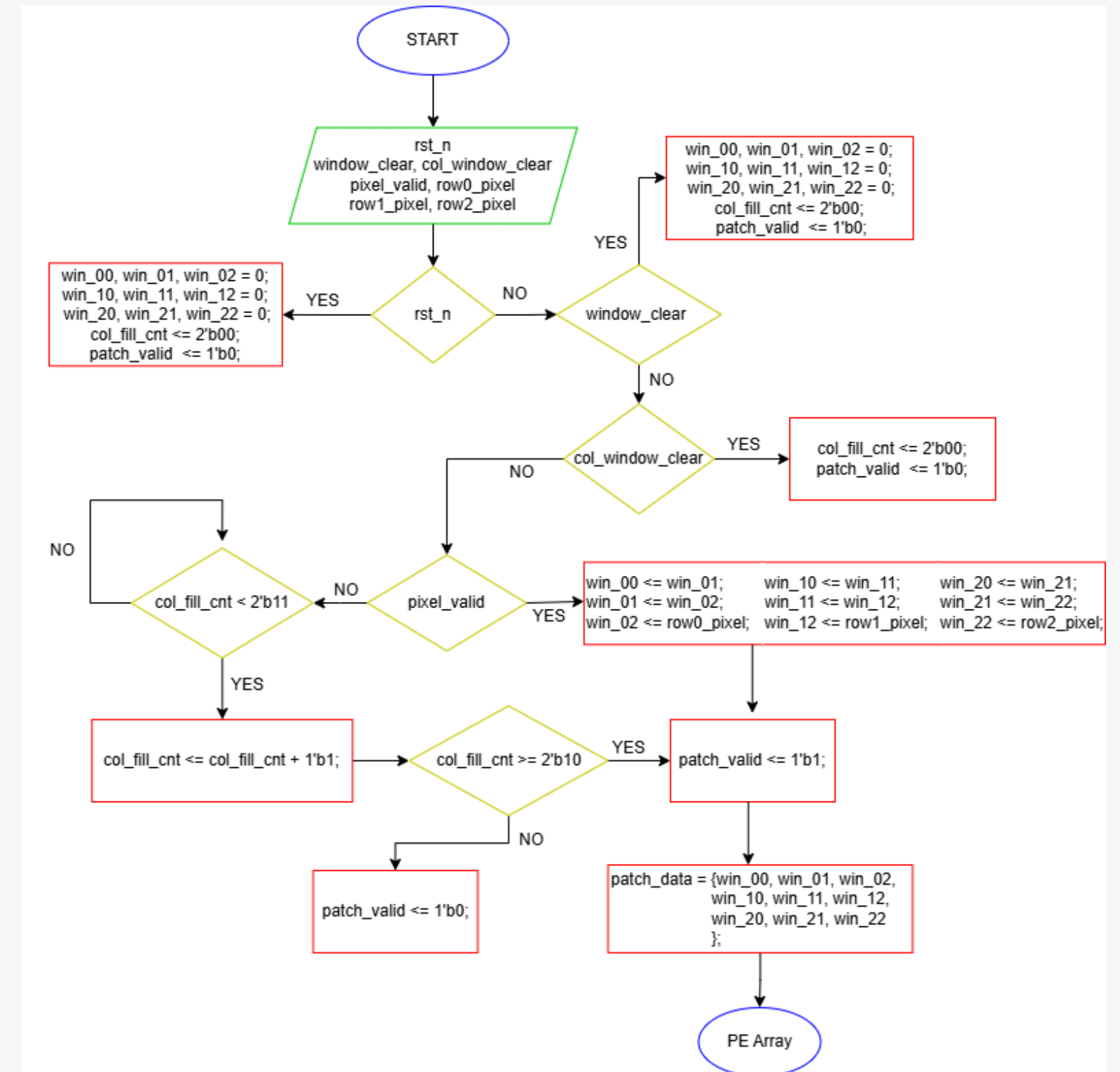
Với mỗi hàng:

win_00 <= win_01

win_01 <= win_02

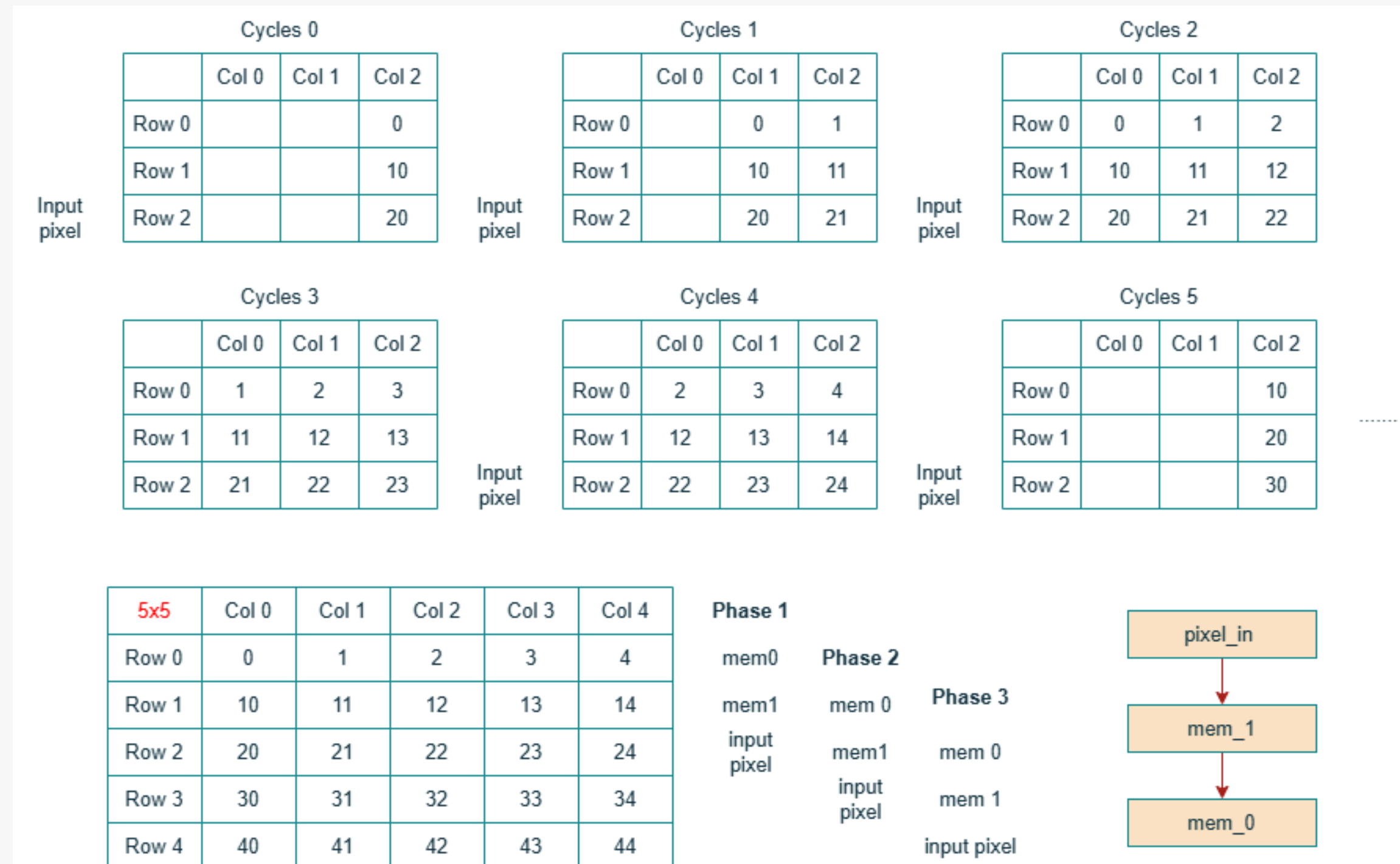
win_02 <= row0_pixel

Tương tự cho hàng 1 và hàng 2.



Sliding Window Generator

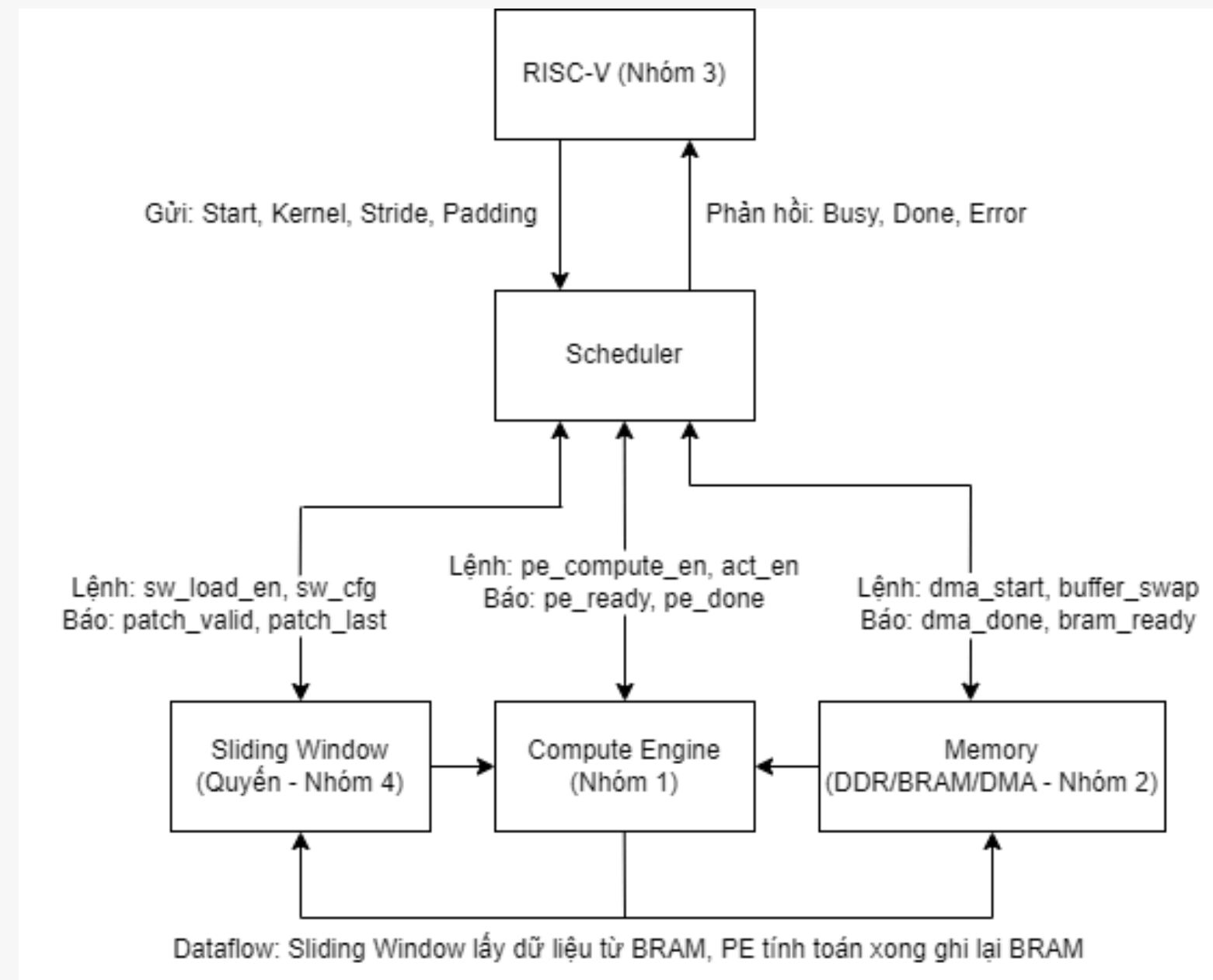
2.2. Nguyên lý hoạt động của Window Generator



Scheduler

1. Kiến trúc tổng thể

- Trung tâm điều phối: Quản lý toàn bộ vòng đời của mạng CNN, giao tiếp với các phân hệ thông qua cơ chế Handshake (phát lệnh Kích hoạt, chờ chờ Hoàn thành).
- Tách biệt Control & Data: Scheduler chỉ xử lý tín hiệu điều khiển (Control Path). Dòng dữ liệu (Data Path) chảy độc lập ở tầng dưới giúp tối ưu thông lượng phần cứng.
- Cầu nối Phần mềm - Phần cứng: Đóng vai trò là thanh ghi giao tiếp (AXI4-Lite), nhận tham số cấu hình (Kernel, Stride...) từ RISC-V và phản hồi trạng thái hệ thống.



Scheduler

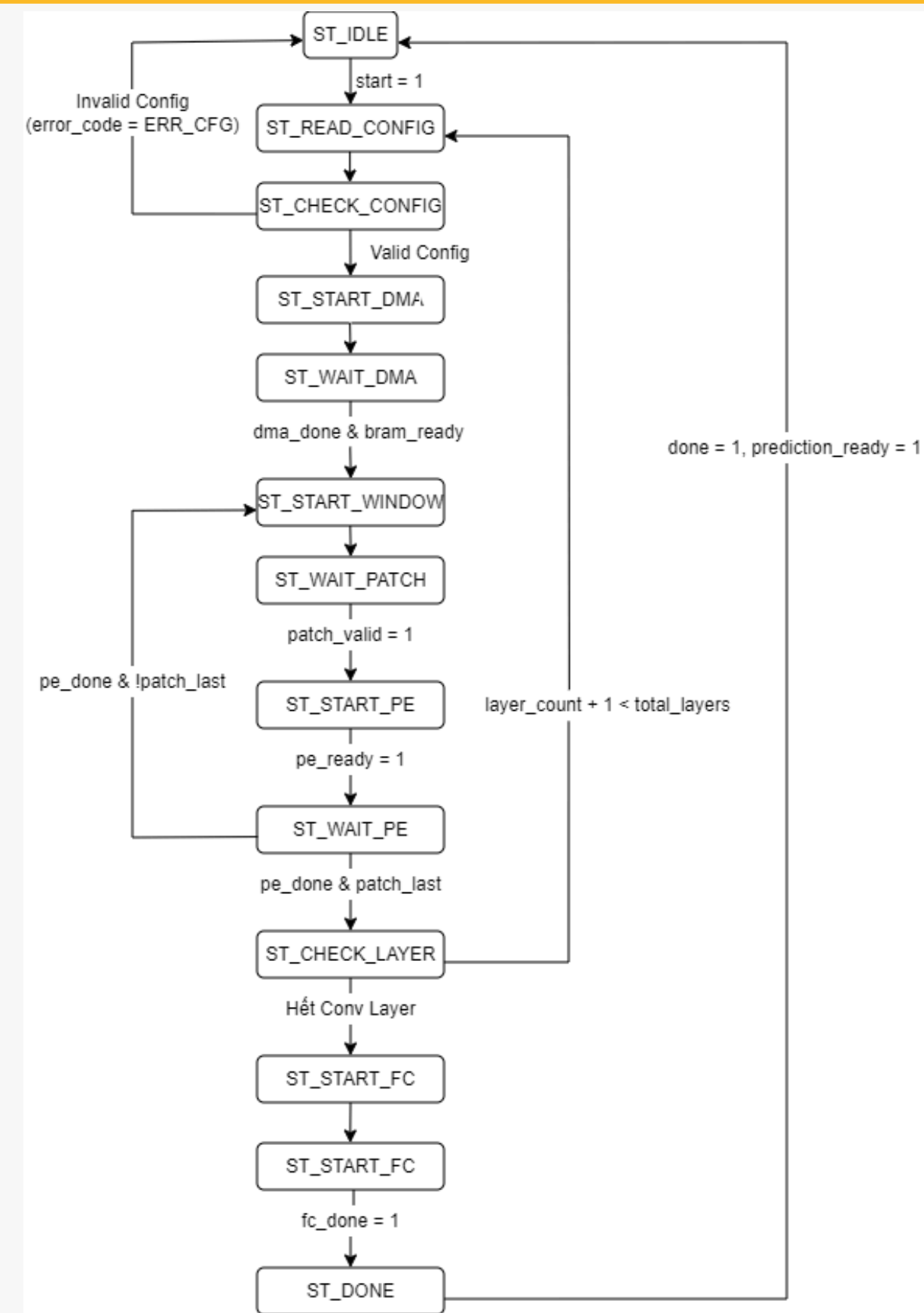
2. Lưu đồ hoạt động

Đồng bộ bằng cơ chế Handshake: Các trạng thái chờ (Wait States) sử dụng tín hiệu valid/ready để đảm bảo luồng dữ liệu giữa Sliding Window, PE Array và Memory không bị xung đột hay rớt nhịp.

Xử lý vòng lặp kép (Dual-Loop):

- Vòng lặp Patch: Kích hoạt đọc và tính toán Patch liên tục cho đến khi kết thúc Feature Map hiện tại.
- Vòng lặp Layer: Tự động kiểm tra và chuyển tiếp sang cấu hình lớp mạng tiếp theo (từ Conv sang FC).

Cơ chế bắt lỗi an toàn: Tự động rẽ nhánh về trạng thái nghỉ (ST_IDLE) nếu phát hiện thông số cấu hình từ RISC-V gửi xuống không hợp lệ.



Scheduler

3. Sơ đồ tuần tự

